

Embedded Systems Architectures - Laboratory Guide 6

SPI Interface and SD Card

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Configuration and use of an SPI bus
- Reading and writing to a microSD card via the SPI bus
- Integration of the DHT20 multi-sensor for temperature and humidity reading and recording to a file on the microSD card

2. Introduction

The Serial Peripheral Interface (SPI) bus is essential in the embedded systems ecosystem, enabling fast and efficient communication between a microcontroller and a wide variety of peripherals – from sensors and actuators to communication modules and storage devices. Its structural simplicity, based on a few signal lines, makes it particularly suitable for integrating multiple components into a single system, maintaining low cost and high reliability.

On platforms like the ESP32-C6, SPI is frequently used to connect measurement sensors, interface controllers, expansion modules, and memory cards. This versatility makes it a strategic tool for developing IoT and automation applications, where it is common to combine data acquisition, local processing, and storage.

The use of SD cards via SPI adds an important layer of functionality: persistent local storage. This allows, for example, saving sensor logs, operating events, or system configurations, even in environments without connectivity. Furthermore, SPI access simplifies integrating recording and storage solutions into industrial research or

development projects, without the need for additional complex hardware. Mastering SPI, including read and write operations on SD cards, is therefore a fundamental step in building robust, scalable, and autonomous embedded systems that operate efficiently and reliably in real-world scenarios.

In this class, we will explore using the SPI protocol to read and write data to an SD card.

3. Practical Work

The practical work in this class is based on the Reference API for the ESP32-C6 peripherals and the ESP32-C6 Technical Reference Manual: https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf

Additionally, the technical data sheets for the DHT20 sensor and the Adafruit 160x80 0.96" display, available on the course webpage, should be consulted.

Warning! This lab handout assumes that the software device driver for the DHT20 sensor was already developed and validated.

3.1 Connection of the integrated display / card reader with the ESP32-C6 kit

The first step in this practical work is to connect the display to the ESP32-C6 development kit. As shown in the display's technical data sheet, it includes an SD card reader, which will be fundamental for this practical lesson. It is possible to visualize the TFT and the card reader in Figure 1, with pinout information available on the PCB silkscreen layer.

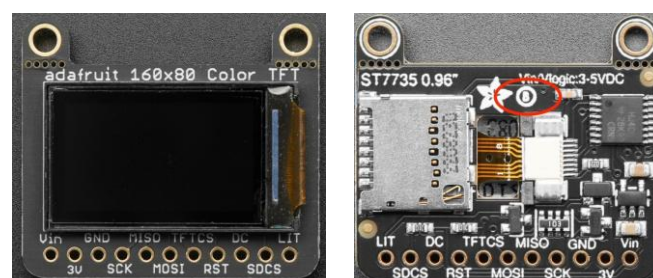


Figure 1 - Adafruit Display (TFT View – left; SD card reader view - right).

As can be seen, this display has a set of pins that allows it to operate simultaneously as a display and as an SD card reader. The available pins for connection and their functions can be summarized as follows:

- **Vin (+3V to +5V)** – power supply pin; must be connected to +3V to +5 V DC.
- **3V** – +3V (approx.) output from the display integrated voltage regulator (leave unconnected).
- **GND** – ground pin, common to power supply and signals.
- **SCK** – SPI clock input pin (accepts voltage levels between +3V and +5 V).
- **MISO** – SPI Master Input (microcontroller in) Slave Output (module out) pin, used by the SD card, not used by the TFT display - write only (3.3 V logic output).
- **MOSI** – SPI Master Output (microcontroller out) Slave Input (module in) pin, used to send data from the microcontroller to the SD card or TFT display (accepts voltage levels between +3V and +5 V).
- **TFTCS** – SPI chip select pin of the TFT display, used to write into the display (accepts voltage levels between +3V and +5 V).
- **RST** – TFT display reset pin, connect to ground to reset the display, includes an automatic reset circuit when power is turned on (accepts voltage levels between +3V and +5 V).
- **DC (Data/Command)** – TFT display data or command selector pin (accepts voltage levels between +3V and +5 V).
- **SDCS (SD Card CS)** – SPI chip select pin of the SD card, used to read/write data from/to the card (accepts voltage levels between +3V and +5 V).
- **LIT** – PWM input pin for backlight control of the display, by default it is at a high level (backlight on), it can be controlled by PWM signal or set to a low level to turn off the backlight (accepts voltage levels between +3V and +5 V).

In this lab, we will only use the SD card reader feature. For this reason, the TFTCS pin must be connected to 3.3V to keep the display inactive during the various exercises. Figure 2 shows the necessary connections enabling the card reader on the ESP32-C6

development kit. Proceed with the circuit assembly, paying close attention to the connections made.

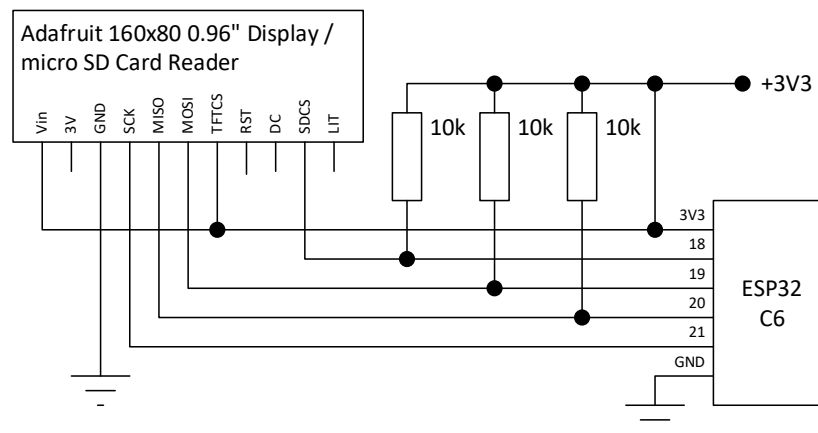


Figure 2 - SPI connection circuit to the SD card reader.

After assembling the circuit, insert the microSD card into the reader slot.

3.2 Test application for reading/writing SD cards

Using the ESP-IDF extension wizard, create a new project based on the example: *storage* -> *sd_card* -> *sdspi*. Then, compile the example and download it to the board.

You should see the following information in the terminal:

```
I (444) main_task: Calling app_main()
I (444) example: Initializing SD card
I (444) example: Using SPI peripheral
I (454) example: Mounting filesystem
I (454) sdspi_transaction: cmd=52, R1 response: command not supported
I (504) sdspi_transaction: cmd=5, R1 response: command not supported
I (524) example: Filesystem mounted
Name: 00000
Type: SDHC
Speed: 20.00 MHz (limit: 20.00 MHz)
Size: 29820MB
CSD: ver=2, sector_size=512, capacity=61071360 read_bl_len=9
SSR: bus_width=1
```

```

I (524) example: Opening file /sdcard/hello.txt
I (544) example: File written
I (544) example: Renaming file /sdcard/hello.txt to /sdcard/foo.txt
I (554) example: Reading file /sdcard/foo.txt
I (554) example: Read from file: 'Hello 00000!'
I (554) example: Opening file /sdcard/nihao.txt
I (574) example: File written
I (574) example: Reading file /sdcard/nihao.txt
I (574) example: Read from file: 'Nihao 00000!'
I (574) example: Card unmounted
I (574) main_task: Returned from app_main()

```

Important! The program will run differently (an error will be displayed) if you have not correctly configured the SPI pins. To correct the error, you must access the SDK Configuration Editor (menuconfig), look for the SD SPI Example Configuration section and assign the pin numbers in a manner consistent with the assembled circuit.

Question 1: Study the SD driver (SD SPI Host Driver) and the support for the file system (FAT Filesystem Support). Analyze the code construction and explain the purpose of this example, identifying the various steps performed.

Question 2: Disconnect the ESP32-C6 development kit from power and remove the microSD card. Using an SD card reader, inspect its contents. To what extent does it reflect the program execution?

3.3 Logging application for recording to SD card

Based on the previous example, the aim now is to develop variations of the program with more useful functionality.

Question 3: Modify the previous program to write the value in milliseconds since the board started to the *log.txt* file, saved at the root of the microSD card, along with a count, in CSV format. The writing on the card should be performed with a period of approximately 1 second.

Run the program, disconnect the board and remove the microSD card, reading its contents on the laptop.

You should observe information similar to that presented below:

```
millis,counter
270,0
1340,1
2400,2
3460,3
4520,4
5580,5
6640,6
```

3.4 Application for reading the log written to the SD card

The goal now is to evolve the previous program to read information from a log file, line by line, rather than write to it.

Question 4: Develop a program that, every second, performs a new reading of a line from the *log.txt* file (stored in the root of the microSD card) and prints its contents to the terminal. Implement the reading functionality in a FreeRTOS task.

The result of the reading should be similar to the one shown below. Note that in the reading process, the file header is ignored.

```
I (446) main_task: Calling app_main()
I (446) sdread: Mounting /sdcard
I (456) sdspi_transaction: cmd=52, R1 response: command not supported
I (496) sdspi_transaction: cmd=5, R1 response: command not supported
I (516) sdread: Mounted. Card: 00000
I (516) main_task: Returned from app_main()
I (516) sdread: CSV: 210,0
I (1516) sdread: CSV: 1250,1
I (2516) sdread: CSV: 2260,2
I (3516) sdread: CSV: 3280,3
I (4516) sdread: CSV: 4300,4
I (5516) sdread: CSV: 5320,5
I (6516) sdread: CSV: 6340,6
I (7516) sdread: CSV: 7360,7
I (8516) sdread: CSV: 8380,8
```

3.5 Integration of temperature and humidity reading (DHT20) with recording on the microSD card

Finally, the aim is to integrate functionalities. In this sense, you should assemble the circuit in Figure 3, which integrates the card reader and the DHT20 temperature and humidity sensor used in the previous class and the corresponding software device driver.

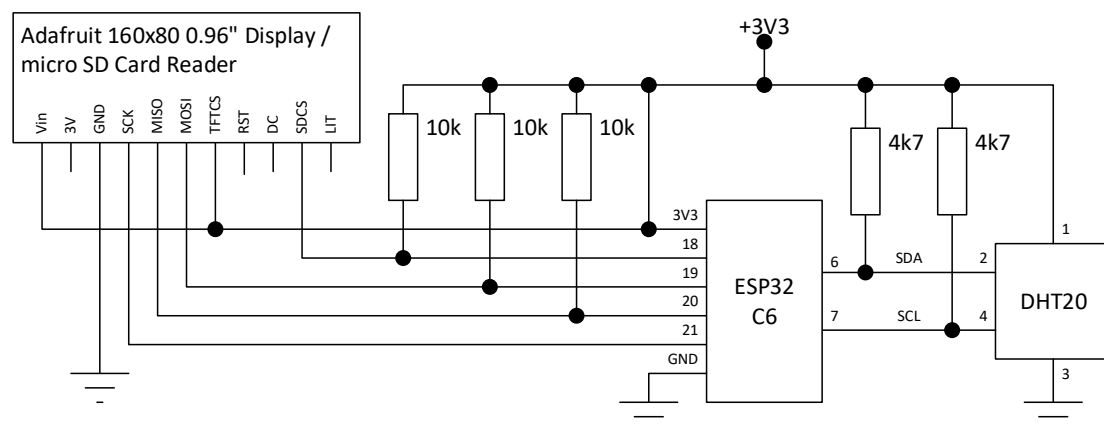


Figure 3 - Circuit for integrating the card reader with the DHT20 sensor.

Question 5: Develop a program that uses the DHT20 sensor to read temperature and humidity every second, saving this information to the *log.txt* file (stored at the root of the microSD card). The functionality of reading and writing to the card should be implemented in a FreeRTOS task. With each write to the card, the elapsed time since the board was powered on, the count value, the temperature, and the humidity reading should be printed to the console.

Test your program on the board, and you should obtain a result similar to the following in the console:

```
I (350) sdlog: Mounted. Card: 00000
I (350) sdlog: I2C initialized successfully for DHT20
I (350) main_task: Returned from app_main()
I (440) sdlog: Logged data: time=170 ms, counter=0, temp=23.71 C,
hum=74.96 %
I (1500) sdlog: Logged data: time=1230 ms, counter=1, temp=23.72 C,
hum=74.94 %
```

```
I (2560) sdlog: Logged data: time=2290 ms, counter=2, temp=23.73 C,  
hum=74.91 %  
I (3620) sdlog: Logged data: time=3350 ms, counter=3, temp=23.73 C,  
hum=74.86 %  
I (4680) sdlog: Logged data: time=4410 ms, counter=4, temp=23.71 C,  
hum=74.91 %  
I (5740) sdlog: Logged data: time=5470 ms, counter=5, temp=23.71 C,  
hum=74.90 %
```

Question 6: Disconnect the board and remove the microSD card. Check the contents of the *log.txt* file, which should look something like this:

```
millis,counter, temperature, humidity  
270,0, 23.69, 74.71  
1340,1, 23.68, 74.77  
2400,2, 23.69, 74.84  
3460,3, 23.69, 74.89  
4520,4, 23.69, 74.92  
5580,5, 23.71, 74.95
```

Note: The examples shown above are inconsistent in their information, as they were collected at different times. They serve only as a pedagogical illustration of the format of the information to be obtained.